

Algoritmos randomizados

Sebastián Torrealba

sebastian.torrealba@sansano.usm.cl



Requisitos

Se asume familiaridad con:

- **C++ y la STL** (`vector` , `sort` , `lower_bound` / `upper_bound` , `shuffle`).
- **Complejidad asintótica** $O(\cdot)$, **valor esperado** y series geométricas.
- **Probabilidad básica**: eventos independientes, probabilidad de la unión.
- **Aritmética modular** y exponenciación rápida; BSGS aparece como caja negra.
- **Operaciones de bits** (XOR) y **sumas de prefijos**.
- **Producto matriz-vector** y su costo $O(n^2)$.
- **Geometría básica**: colinealidad de tres puntos con producto cruz entero.
- **Grafos**: nodos, aristas y conexidad; la búsqueda local camina por un grafo implícito.

No se requiere experiencia previa con algoritmos aleatorizados.



Mapa de la clase



El azar como herramienta, un pilar por **tipo de problema**; en los tres, el mismo hilo:
 $\mathbb{E}[\text{intentos}] = 1/p$.



¿Por qué aleatorizar?

elección fija

quicksort

pivote: siempre el primero



hay un peor caso fijo

input ordenado $\rightarrow O(n^2)$

cada vez, reproducible

vs

elección al azar

quicksort



pivote: uniforme al azar



ningún peor caso fijo

todo input $\rightarrow E = O(n \log n)$

la esperanza es sobre tus tiradas

La moneda **elimina el peor caso fijo**: ningún input pierde siempre, el costo pasa a ser **esperado**.



El patrón de la clase



$$E[\text{intentos}] = 1/p$$

En cada problema: ¿qué se muestrea? y ¿cuál es la cota p ?



Derivación: $\mathbb{E}[\text{intentos}] = 1/p$

Sea N el número de intentos hasta el primer éxito, con $P(\text{éxito}) = p$ en cada intento, independientes. Fallar los primeros k intentos tiene probabilidad

$$P(N > k) = (1 - p)^k.$$

Para una variable entera no negativa, $\mathbb{E}[N] = \sum_{k \geq 0} P(N > k)$, y la suma es una serie geométrica:

$$\mathbb{E}[N] = \sum_{k \geq 0} (1 - p)^k = \frac{1}{1 - (1 - p)} = \frac{1}{p}.$$

Si cada intento cuesta T : $\mathbb{E}[\text{tiempo}] = T/p$.



Dos tipos de problema

puedes verificar barato

salida

siempre correcta ✓

tiempo · **aquí vive el azar**



aleatorio · $E = T/p$

estrategia · **repetir hasta acertar**

error = 0

verificar es caro

salida · **aquí vive el azar**



puede fallar

tiempo

fijo · K iteraciones

estrategia · **iterar K veces**

error $\leq (1-p)^K$

En esta clase: mayoritario, CSES 3422 y Balloon Darts **verifican barato**; Farming Mars, Freivalds y el hashing XOR **acotan el error**.

Nombres clásicos, por si los ven en la literatura: *Las Vegas* (izquierda) y *Monte Carlo* (derecha).

Pilar 1: Construcción

1 · Construcción

2 · Verificación

3 · Optimización



Problema: elemento mayoritario

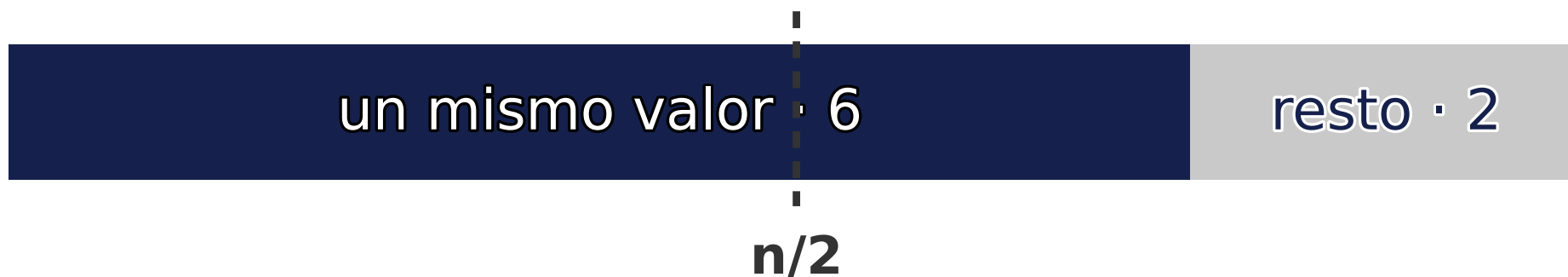
Un arreglo de n elementos contiene un valor que aparece **más de $n/2$ veces**. Hay que encontrarlo.

Es un problema básico: se resuelve con un **mapa de frecuencias** u **ordenando**, e incluso existe la solución determinista de Boyer-Moore en $O(n)$.

Lo usamos como calentamiento: ¿cómo se ve bajo la **lente aleatoria**?



Mayoritario: ¿qué se busca?



Un valor ocupa **más de $n/2$** casillas: es el **mayoritario**, y hay que encontrarlo.

¿Cómo lo encontrarías usando aleatoriedad?



Mayoritario: muestrear y verificar

El mayoritario aparece $f > \frac{n}{2}$ veces, así que un índice al azar cae sobre él con

$$p = \frac{f}{n} > \frac{1}{2}.$$

Tomamos $a[i]$ como **candidato** y lo **verificamos** contando sus apariciones en $O(n)$.

¿Puede este método devolver una respuesta incorrecta?



Mayoritario: muestrear un índice



índice al azar $\cdot p > \frac{1}{2}$



Candidato al azar, verificación exacta en $O(n)$: $\mathbb{E}[\text{intentos}] = 1/p < 2$.



Mayoritario: análisis

La verificación en $O(n)$ es exacta: nunca devolvemos un valor incorrecto, lo único aleatorio es el **tiempo**. Con $p > 1/2$:

$$\mathbb{E}[\text{intentos}] = \frac{1}{p} < 2, \quad \mathbb{E}[\text{tiempo}] < 2n.$$

Ejercicio. Arreglo de 100 elementos: 65 son **A**, 35 son **B**.

(a) $P(\text{elegir } A)$; (b) $\mathbb{E}[\text{intentos}]$; (c) operaciones esperadas, si cada intento cuenta los 100 elementos.

Para pasar a tiempo, la regla de siempre: 10^8 operaciones $\approx$ 1 segundo.



Problema: Farming Mars

Se dispone de n acres, cada uno con un pH, y m consultas $[l, r]$. Por consulta: ¿una **mayoría estricta** ($> \frac{r-l+1}{2}$) de los acres en $[l, r]$ comparten el **mismo** pH? (no necesitan ser contiguos).

A la mala: recorrer $[l, r]$ y contar en cada consulta cuesta $O(n)$; con $n, m \leq 10^4$, el total $O(nm)$ llega a 10^8 .

Es la **versión en rango** del mayoritario.



Farming Mars: ¿qué pide cada consulta?



┌──────────────────┐
consulta $[l_1, r_1]$ · ¿mayoría estricta?

┌──────────────────┐
consulta $[l_2, r_2]$ · ¿mayoría estricta?

⋮ hasta $m = 10^4$ consultas

Por consulta $[l, r]$: ¿algún pH en **más de la mitad** de esos acres (no contiguos)?

┌ ¿Cómo extender la idea del mayoritario a un rango $[l, r]$ con muchas consultas?



Farming Mars: muestreo y verificación

Idea. Muestrear ~ 30 índices al azar dentro de $[l, r]$; si hay mayoría, cada muestra la encuentra con probabilidad $> 1/2$.

Para contar las apariciones de un valor v dentro de $[l, r]$:

- Preproceso $O(n)$: para cada valor, su **lista ordenada de posiciones**.
- Por candidato: `upper_bound - lower_bound` sobre esa lista $\rightarrow$ cuenta en $O(\log n)$.

Costo por consulta: $O(K \log n)$. Se compara contra $\lfloor (r - l + 1)/2 \rfloor + 1$.



Farming Mars: la solución en una imagen



muestrear $K \approx 30$ índices al azar en $[l, r]$

posiciones del candidato 3, ordenadas:



$O(\log n)$

$\text{upper_bound}(r) - \text{lower_bound}(l) = 5$

Muestrear en $[l, r]$ y contar con búsqueda binaria: $O(K \log n)$ por consulta.



falso positivo = 0

«usable» solo tras verificar
la mayoría: determinista

falso negativo

hay mayoría, pero las K
muestras no la tocan

$$P(\text{falso negativo}) = (1 - p)^K \leq 2^{-K} \quad (p \geq \tfrac{1}{2}).$$

Ejercicio. $K = 10 \Rightarrow P(\text{error}) < 2^{-10} \approx 10^{-3}$. ¿Qué K da $\leq 10^{-6}$?

Queremos $2^{-K} \leq 10^{-6}$. Tomando logaritmos: $K \geq 6 \log_2 10 \approx 19.9$, es decir, $K \geq 20$.

K	10	20	30
$P(\text{error}) \leq$	$\approx 10^{-3}$	$\approx 10^{-6}$	$\approx 10^{-9}$

Cada muestra extra **duplica** la confianza: la amplificación es exponencial y cuesta solo $O(\log n)$ por muestra.

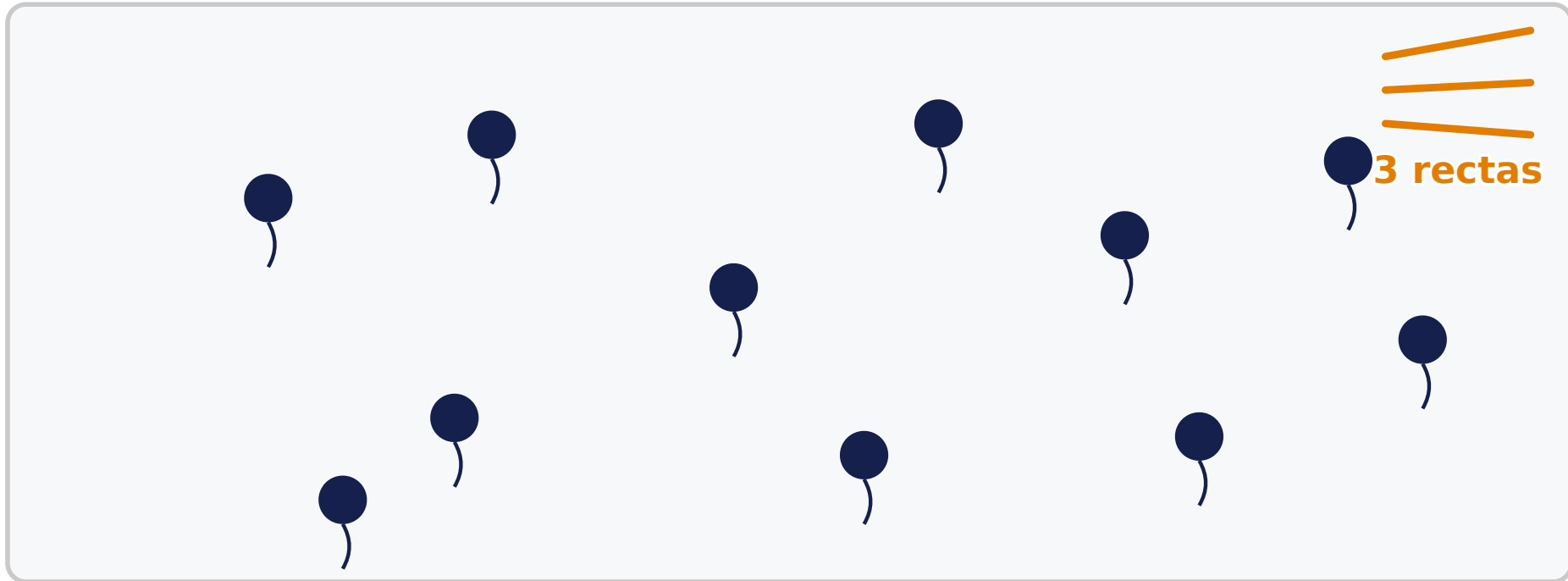


Problema: Balloon Darts

n puntos en el plano. Se dispone de **3 rectas** infinitas. ¿Es posible **cubrir todos** los puntos con esas 3 rectas?

A la mala: asignar cada punto a una de las 3 rectas da 3^n combinaciones; con $n = 10^4$, imposible.

Balloon Darts: ¿bastan 3 rectas?



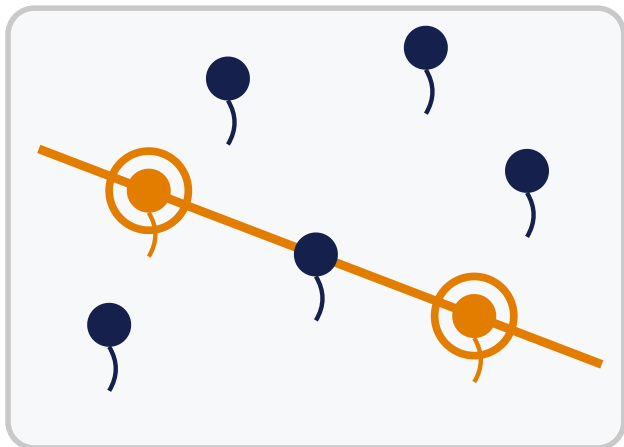
¿Se pueden **cubrir los n puntos** con 3 rectas infinitas? Decisión: sí / no.

¿Cómo elegir las 3 rectas usando aleatoriedad?

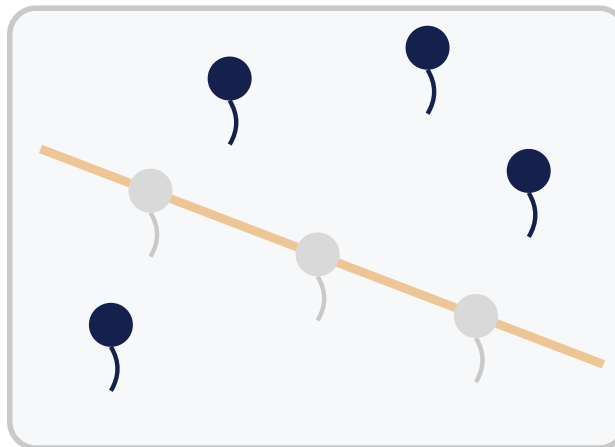


Balloon Darts: idea aleatoria

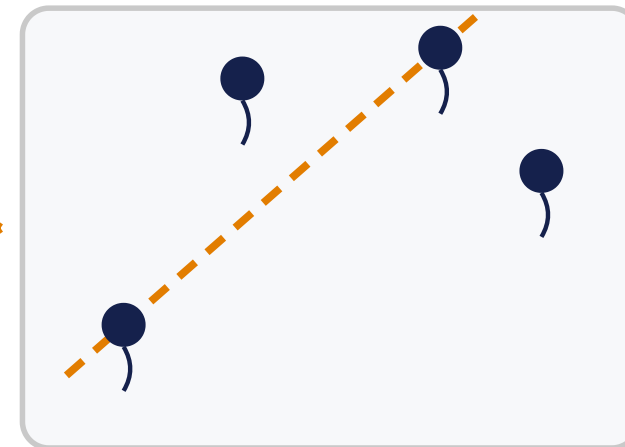
1 · dos puntos al azar



2 · eliminar cubiertos



3 · recurrir con k-1



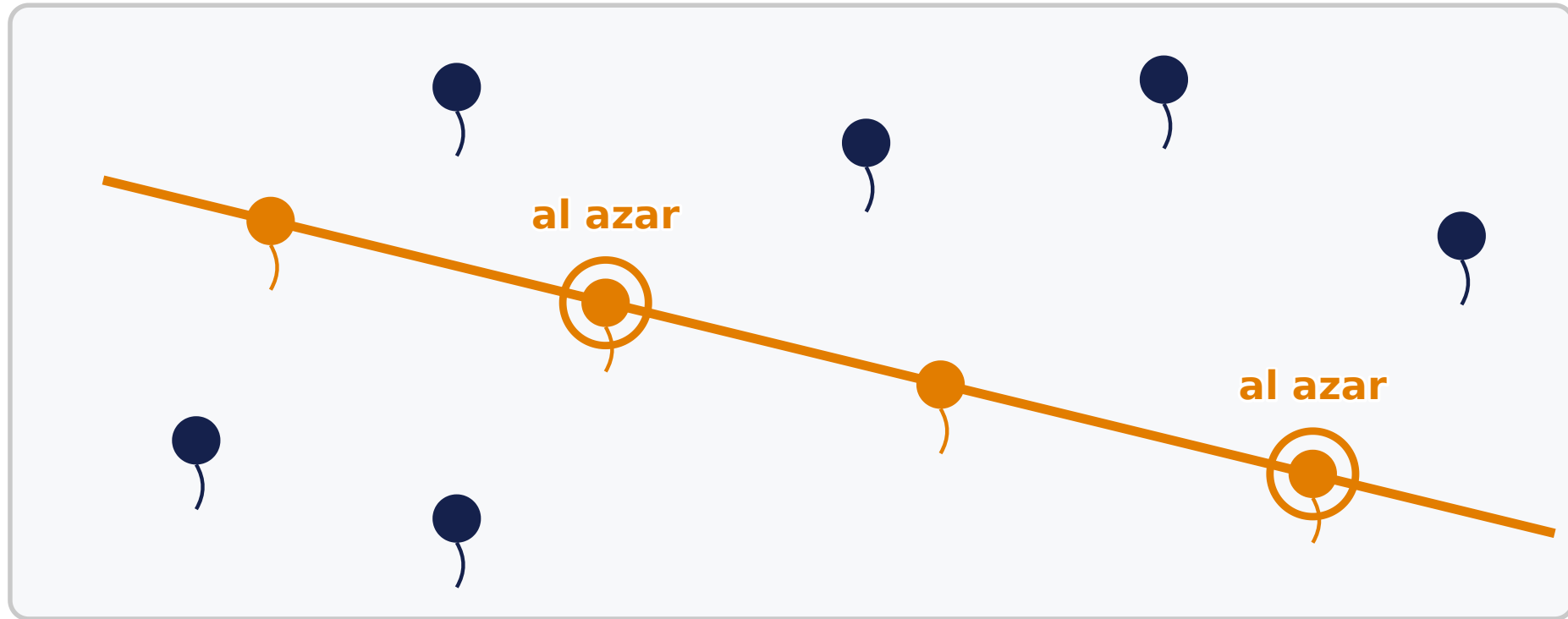
si falla → reintentar

Óptimo de k rectas → fracciones g_i ($\sum_i g_i = 1$); dos puntos al azar comparten recta:

$$P \geq \sum_i g_i^2 \geq \frac{(\sum_i g_i)^2}{k} = \frac{1}{k} \quad (\text{Cauchy-Schwarz}).$$



Balloon Darts: la solución en una imagen



$$k = 3 \cdot P \geq \frac{1}{3}$$



$$k = 2 \cdot P \geq \frac{1}{2}$$



$$k = 1 \cdot P = 1$$

Dos puntos al azar dan una recta del óptimo con $P \geq 1/k$: eliminar y recurrir.



Balloon Darts: análisis

$$P(\text{éxito}) \geq \underbrace{\frac{1}{3}}_{k=3} \cdot \underbrace{\frac{1}{2}}_{k=2} \cdot \underbrace{1}_{k=1} = \frac{1}{6}$$

$$P(\text{fallar las } T \text{ corridas}) \leq \left(\frac{5}{6}\right)^T, \quad \left(\frac{5}{6}\right)^{80} < 10^{-6}$$

veredicto «sí»

cobertura verificada en $O(n)$

error = 0

veredicto «imposible»

error $\leq (5/6)^T$ tras T corridas

todo el error vive aquí

La colinealidad debe calcularse con **aritmética entera**: con coordenadas $\sim 10^9$, `double` produce falsos colineales.



Problema: CSES 3422 (Third Permutation)

Dadas permutaciones a, b con $a_i \neq b_i$ en toda posición, construir c con

$$c_i \neq a_i \quad \mathbf{y} \quad c_i \neq b_i \quad \text{para todo } i.$$

A la mala: enumerar las $n!$ permutaciones es inviable ($n = 10^5$).

En el problema original puede no existir solución (p. ej. $n = 2$): se responde `IMPOSSIBLE`.

<i>a</i>	3	1	4	2	6	5
<i>b</i>	5	2	1	6	3	4
	$\neq$	$\neq$	$\neq$	$\neq$	$\neq$	$\neq$
<i>c</i>	?	?	?	?	?	?

en cada columna, *c* debe evitar 2 valores de *n*

Construir la permutación *c* evitando los **2 valores prohibidos** de cada columna.

¿Cómo construir *c* de forma aleatoria?



Idea. Generar c al azar (`std::shuffle`) y **verificar** las restricciones en $O(n)$. Si falla, repetir.

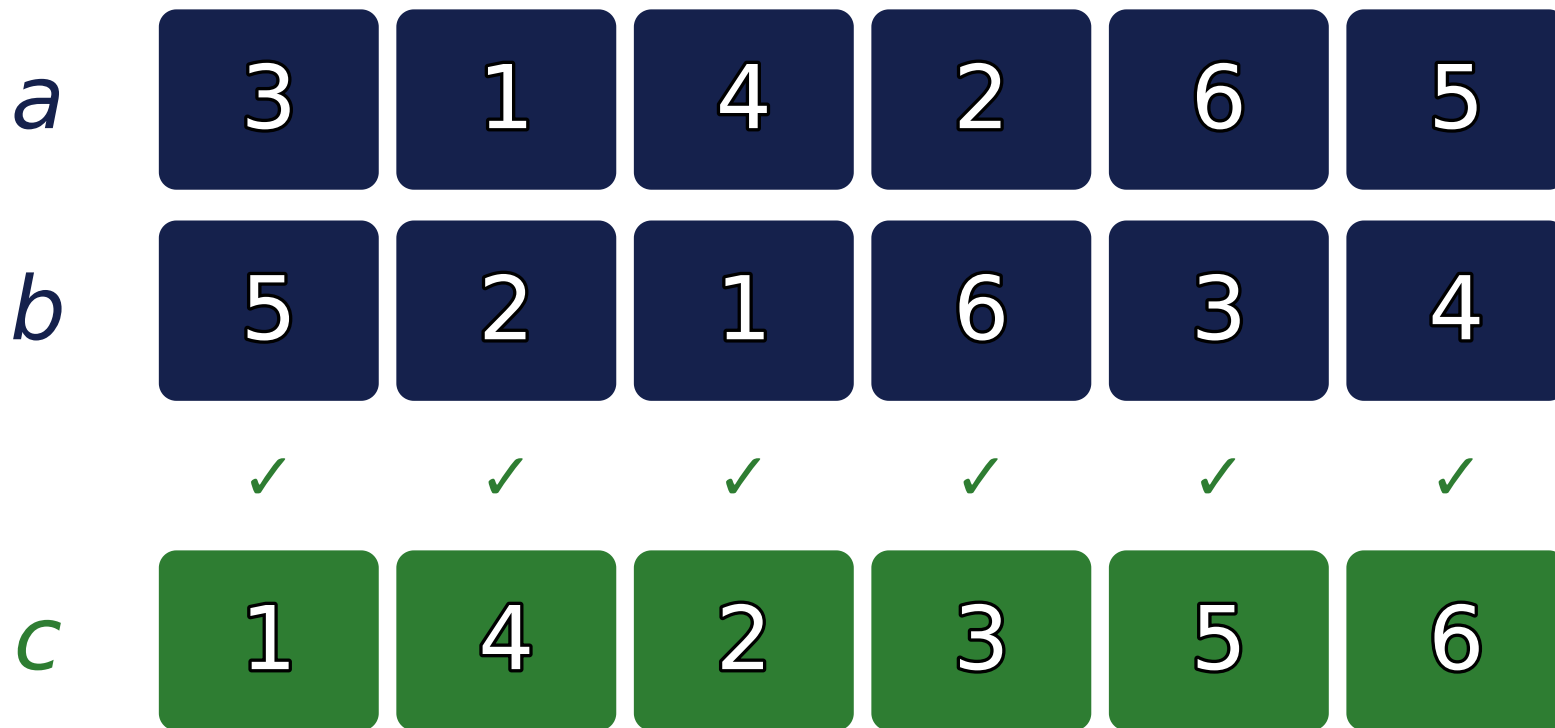
El número esperado de posiciones prohibidas golpeadas es ≈ 2 . Por una aproximación tipo *derangement*:

$$P(\text{éxito por intento}) \approx e^{-2} \approx 13.5\%.$$

- **Estable:** no crece con n .
- $\mathbb{E}[\text{intentos}] \approx e^2 \approx 7.4$, cada uno $O(n) \rightarrow O(n)$ esperado.



CSES 3422: la solución en una imagen



shuffle

$$P(\text{éxito}) \approx e^{-2} \approx 13.5\% \cdot E[\text{intentos}] \approx e^2 \approx 7.4$$

Barajar y verificar en $O(n)$: $P(\text{éxito}) \approx e^{-2}$, estable en n .

Pilar 2: Verificación

1 · Construcción

2 · **Verificación**

3 · Optimización



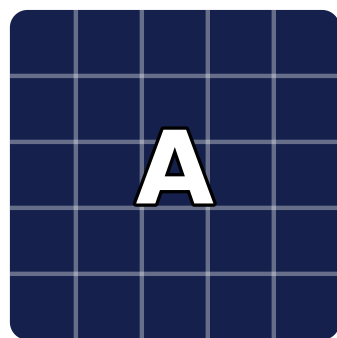
Problema: ¿es correcto $A \cdot B = C$?

Se entregan tres matrices $n \times n$ con la afirmación $A \cdot B = C$, y hay que **verificarla**.

Recomputar $A \cdot B$ cuesta $O(n^3)$ (u $O(n^{2.37})$ con Strassen). Con $n = 2000$ son $\sim 8 \times 10^9$ operaciones, solo para comprobar lo que alguien ya afirma.

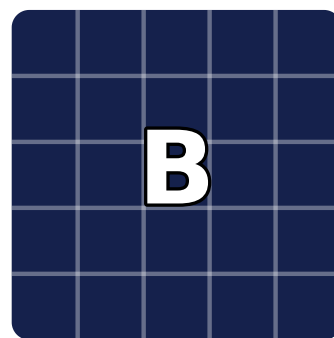


Freivalds: ¿es correcto $A \cdot B = C$?



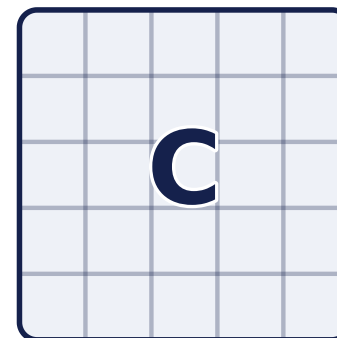
$n \times n$

.



$n \times n$

?



$n \times n$

recomputar $A \cdot B$: $O(n^3) \approx 8 \cdot 10^9$ operaciones con $n = 2000$

Afirman que $A \cdot B = C$: hay que **verificarlo** sin pagar el producto completo.

¿Se puede verificar en solo $O(n^2)$ admitiendo aleatoriedad?



Freivalds: el algoritmo

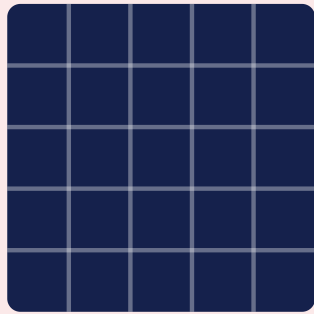
Elegir un vector aleatorio $r \in \{0, 1\}^n$ y comprobar

$$A (B r) \stackrel{?}{=} C r \quad \text{en } O(n^2).$$

Son dos productos matriz-vector, cada uno $O(n^2)$; nunca se forma $A \cdot B$.

Si $A \cdot B = C$, la igualdad se cumple siempre (sin falsos negativos en ese caso).

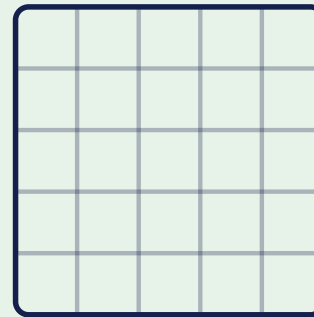
recomputar $A \cdot B$



$O(n^3)$

$\approx 8 \cdot 10^9$ operaciones

$A(Br) \stackrel{?}{=} Cr$



.

=



r al azar

$O(n^2)$

dos productos matriz·vector

Un vector aleatorio r reduce la verificación a dos productos matriz-vector.



Freivalds: un ejemplo

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad C = \begin{pmatrix} 19 & 22 \\ 43 & \mathbf{51} \end{pmatrix}$$

Lo correcto es 50, no 51: la matriz C está mal. Tomemos $r = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$:

$$B r = \begin{pmatrix} 6 \\ 8 \end{pmatrix}, \quad A(Br) = \begin{pmatrix} 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 22 \\ 50 \end{pmatrix}, \quad C r = \begin{pmatrix} 22 \\ 51 \end{pmatrix}.$$

Como $50 \neq 51$, el error se detecta **sin** calcular $A \cdot B$.

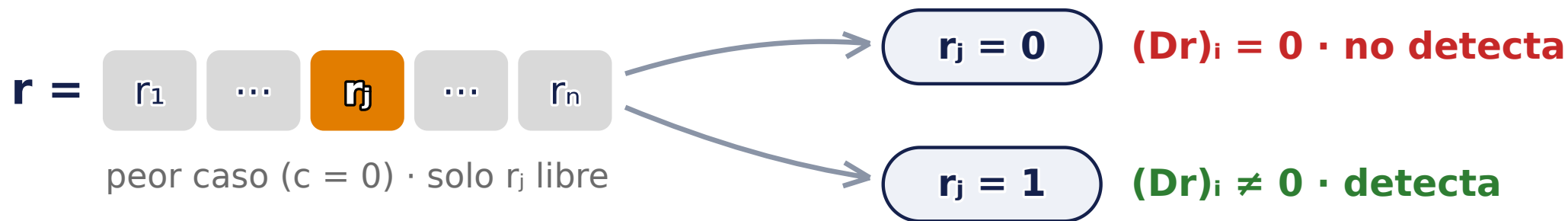
Con $r = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ no se habría visto (el error está en la otra columna): por eso se repite con varios r .



Freivalds: cota de error

Sea $D = AB - C \neq 0$: fijamos $D_{ij} \neq 0$ y condicionamos toda coordenada de r salvo r_j :

$$(Dr)_i = D_{ij} r_j + c.$$



Vale 0 para **a lo sumo uno** de los dos valores $r_j \in \{0, 1\}$:

$$P((Dr)_i = 0) \leq \frac{1}{2} \Rightarrow P(\text{error}) \leq \frac{1}{2} \Rightarrow 2^{-t} \text{ con } t \text{ intentos.}$$

Error **unilateral**. Mejora de $O(n^3)$ a $O(n^2)$: factor $\Theta(n)$.



Problema: ¿es una permutación?



$a[1..3] = \{2, 1, 3\}$ · permutación de $1..3$ ✓

$a[3..6] = \{3, 5, 4, 2\}$ · $k = 4$, falta el 1 ✗

a la mala: revisar el rango completo · $O(n)$ por consulta · $O(n \cdot q)$ total

Dado $a_1, \dots, a_n$ y q consultas (l, r) : decidir si $a_l, \dots, a_r$ es una **permutación** de $\{1, \dots, k\}$, con $k = r - l + 1$.

Lo ingenuo cuesta $O(n)$ por consulta: ¿se puede responder cada una en $O(1)$, admitiendo aleatoriedad?



Repaso: propiedades del XOR

$$\begin{array}{cccc} 1 & 0 & 1 & 1 \\ \oplus & 0 & 1 & 1 \\ \hline 1 & 1 & 0 & 1 \end{array} \begin{array}{l} = 11 \\ = 6 \\ = 13 \end{array}$$

cada columna por separado: suma mód 2, sin acarreo

$$a \oplus b = b \oplus a$$

conmuta

$$(a \oplus b) \oplus c = a \oplus (b \oplus c)$$

asocia

$$a \oplus 0 = a$$

el 0 es neutro

$$a \oplus a = 0$$

su propio inverso

$$(a \oplus b) \oplus b = a \cdot \text{aplicar } \oplus b \text{ de nuevo lo deshace}$$

Suma **bit a bit sin acarreo**: con enteros de 64 bits son 64 sumas mód 2 en paralelo, en una instrucción.

Lo que usará la clase: el orden no importa y **los pares se cancelan**.



Hashing XOR: una huella para conjuntos

UNA ETIQUETA ALEATORIA DE 64 BITS POR VALOR

$h(1) = 5b3a\dots$

$h(2) = c1f7\dots$

$h(3) = 90d4\dots$

$h(4) = 2e8b\dots$

$h(5) = a604\dots$

$\{2, 1, 3\}$



$h(2) \oplus h(1) \oplus h(3)$



huella = 0a19...

$\oplus$ **conmuta: el orden no importa**

$\{2,1,3\}$ y $\{3,2,1\} \rightarrow$ la misma huella

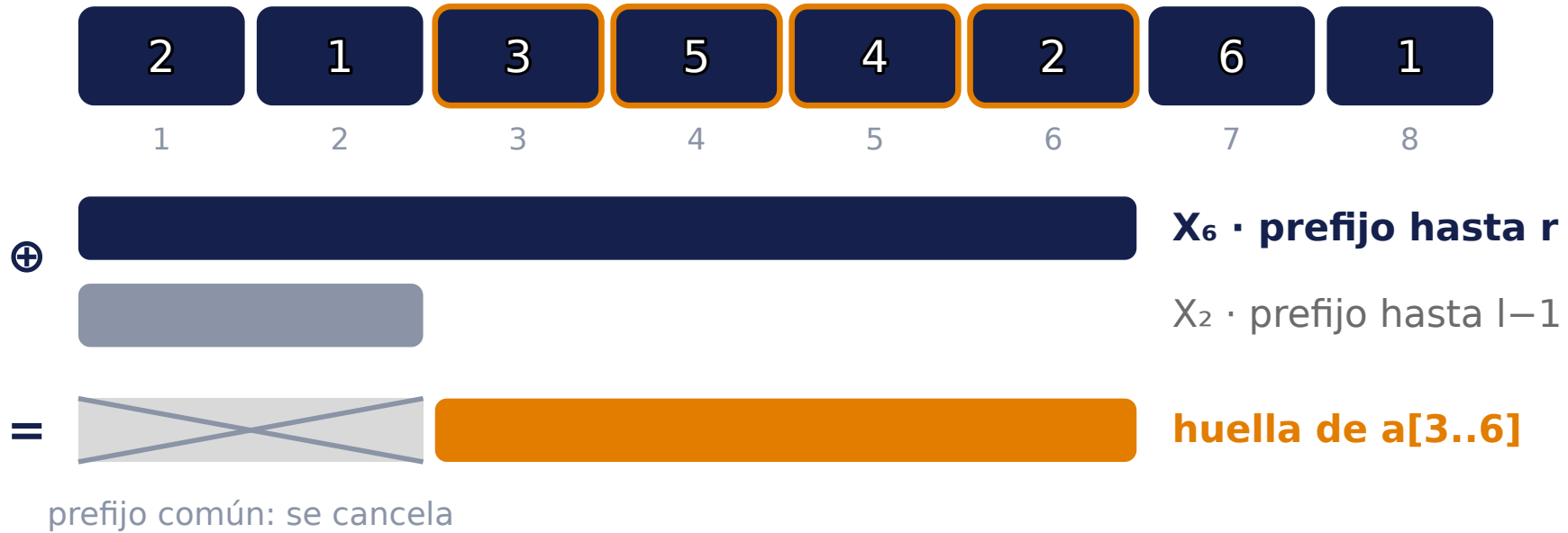
$x \oplus x = 0$: los pares se cancelan

la huella ve cuántas veces, mód 2

Conjuntos iguales $\rightarrow$ huellas iguales, **siempre**; conjuntos distintos $\rightarrow$ huellas distintas, **salvo mala suerte**.



Hashing XOR: la solución en una imagen



$$X_6 \oplus X_2 \stackrel{?}{=} P_4 = h(1) \oplus \dots \oplus h(4)$$

$$X_i = h(a_1) \oplus \dots \oplus h(a_i), \quad \text{aceptar si } X_r \oplus X_{l-1} = P_k, \quad P_k = h(1) \oplus \dots \oplus h(k).$$

Prefijos X_i y objetivos P_k en $O(n)$; después cada consulta es un XOR y una comparación: $O(n + q)$.



Hashing XOR: cota de error

huella($a[l..r]$) $\oplus$ P_k

=

$h(v)$

$\oplus$

$c \cdot$ todo lo demás, fijo

v : un valor descuadrado (paridad distinta a ambos lados) · solo $h(v)$ queda libre

$h(v) = c$

engaña · 1 de 2^{64}

$h(v) \neq c$

detecta · el resto

cuidado: XOR ve paridad, no multiplicidad

$\{2,2,3\}$ y $\{3\}$: misma huella · multiconjuntos $\rightarrow$ sumar

aquí la longitud lo salva

k valores con paridad impar $\rightarrow$ exactamente $1..k$

$$P(\text{engaño}) = 2^{-64} \text{ por consulta} \Rightarrow P(\text{alguna falle}) \leq \frac{q}{2^{64}} \approx 10^{-14} \text{ con } q = 2 \cdot 10^5.$$

Error **unilateral**: una permutación de verdad siempre pasa. La misma jugada de **Freivalds**: comparar huellas aleatorias en vez del objeto completo, ahora para conjuntos.

Para practicar: Codeforces 1175F (contar subpermutaciones) · ABC 250 E (conjuntos de prefijos).



Problema: Power of Three

Dado un entero N , hallar el exponente k con $3^k = N$, o -1 si N no es potencia de 3.

Restricciones (IEEEXtreme 18.0): N tiene hasta **10^7 dígitos**, con 1 segundo de límite; no cabe en ningún entero nativo, y si hay respuesta, k llega hasta $\approx 2 \times 10^7$.

A la mala: ir calculando $3^0, 3^1, 3^2, \dots$ como enteros gigantes y comparar con N es inviable.



Power of Three: ¿es N una potencia de 3 ?

N — gigante: no cabe en 64 bits

1002001220211002...10022110441

¿ existe k con $3^k = N$? (si no, -1)

3^0

3^1

3^2

3^3

3^4

...

las potencias crecen sin límite: comparar enteros gigantes es inviable

Hallar k con $3^k = N$, o responder -1 : N no cabe en ningún entero nativo.

¿Cómo verificarlo sin manejar N completo? ¿Qué aporta trabajar módulo un primo?

Bajo un primo p , la ecuación

$$3^k \equiv N \pmod{p}$$

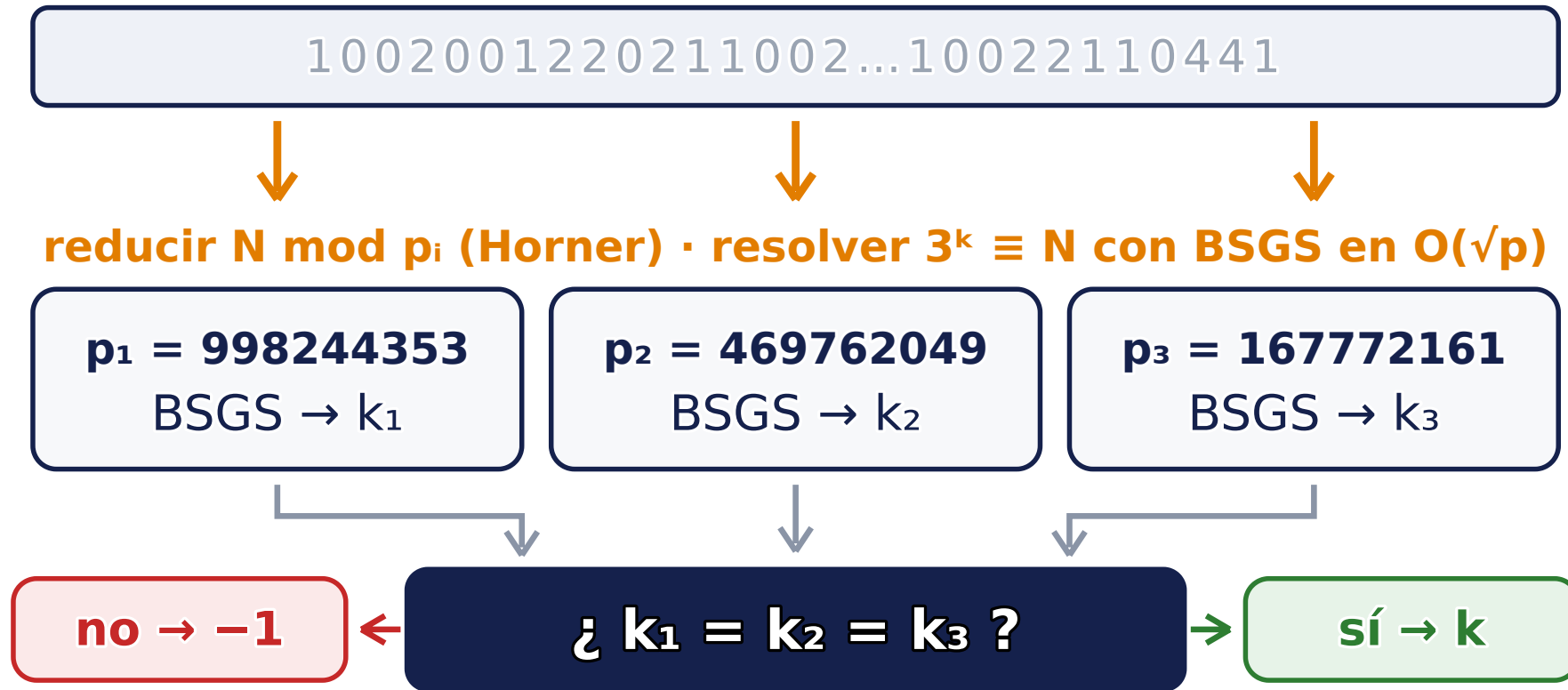
se resuelve con **Baby-Step Giant-Step (BSGS)** en $O(\sqrt{p})$.

Se reduce $N \bmod p$ (leyendo N dígito a dígito) y se aplica BSGS para obtener un candidato k .

Bajo **un** primo puede aparecer un k **espurio**: cumple módulo p sin ser la respuesta real.



Power of Three: tres primos



Aceptar **solo si** $k_1 = k_2 = k_3$: un solo primo puede dar un k espurio.

No es CRT: los k_i viven módulo $p_i - 1$, que comparten factores de 2; la garantía es que **tres módulos independientes coinciden**.

Pilar 3: Optimización

1 · Construcción

2 · Verificación

3 · **Optimización**



El vendedor viajero (TSP)

Dadas n ciudades y la distancia entre cada par, encontrar el **orden** que las visita todas y vuelve al inicio con **distancia total mínima**.

Determinista: probar los $n!$ órdenes, o la DP sobre subconjuntos (**Held-Karp**) en $O(2^n \cdot n^2)$, que alcanza hasta $n \approx 20$.

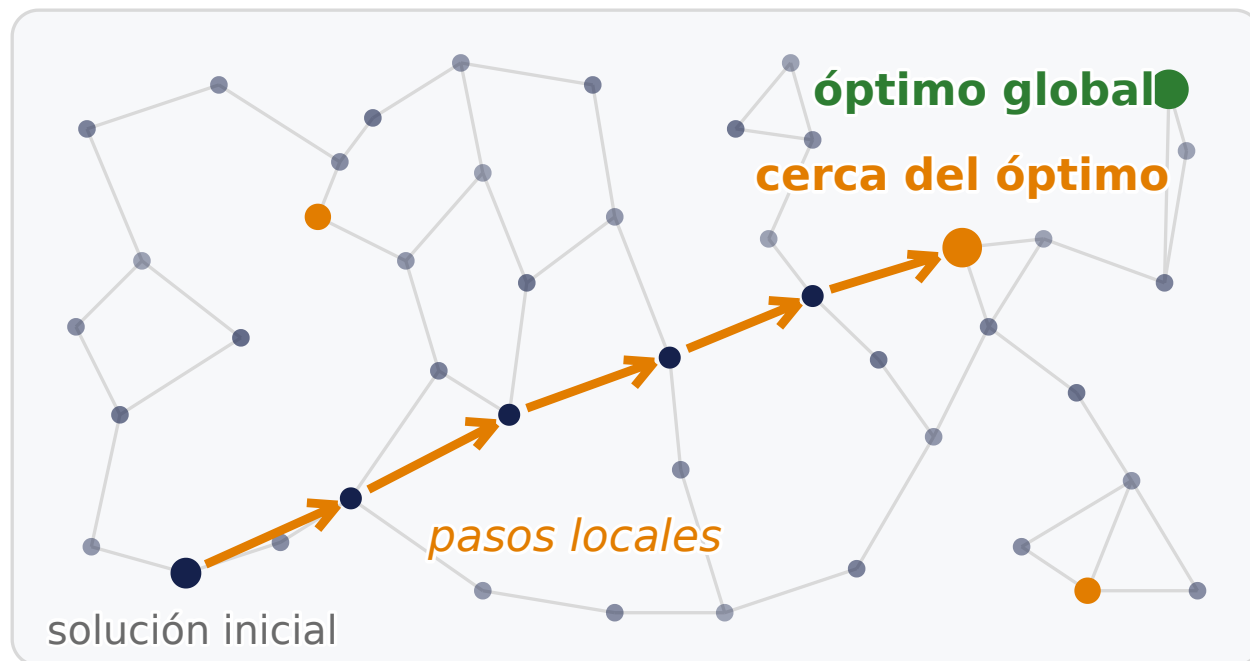
Es **NP-duro**: no se conoce algoritmo polinomial. Con $n = 50$, $2^{50} > 10^{15}$: ni la DP entra en el tiempo.

Si el óptimo exacto es inalcanzable, ¿qué tan cerca se puede quedar en 1 segundo?



Optimización combinatoria

espacio de estados: permutaciones · asignaciones · rutas



a la mala: enumerar

$n!$ rutas del TSP, $n = 50$

$n! \approx 3 \cdot 10^{64}$

más que átomos en la Tierra

greedy $\times$

DP $\times$

la respuesta está cerca
de una solución random

Estrategia con $n \leq 100$: partir de una solución y **mejorarla por pasos locales**.

¿Qué pasa si **solo** aceptamos pasos que mejoran?



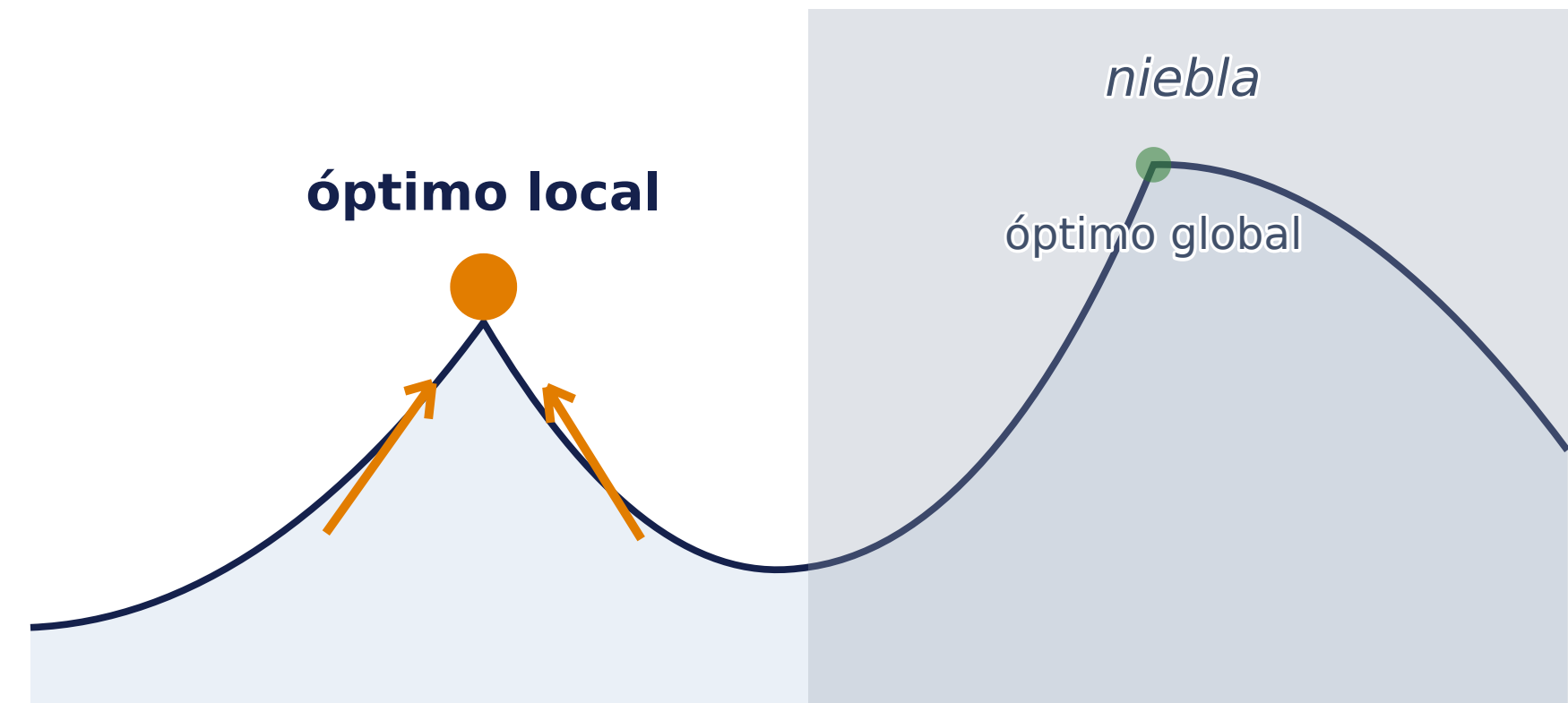
Hill Climbing: la idea



Se le llama **búsqueda local**: nunca miramos todo el mapa, solo lo que está al lado.



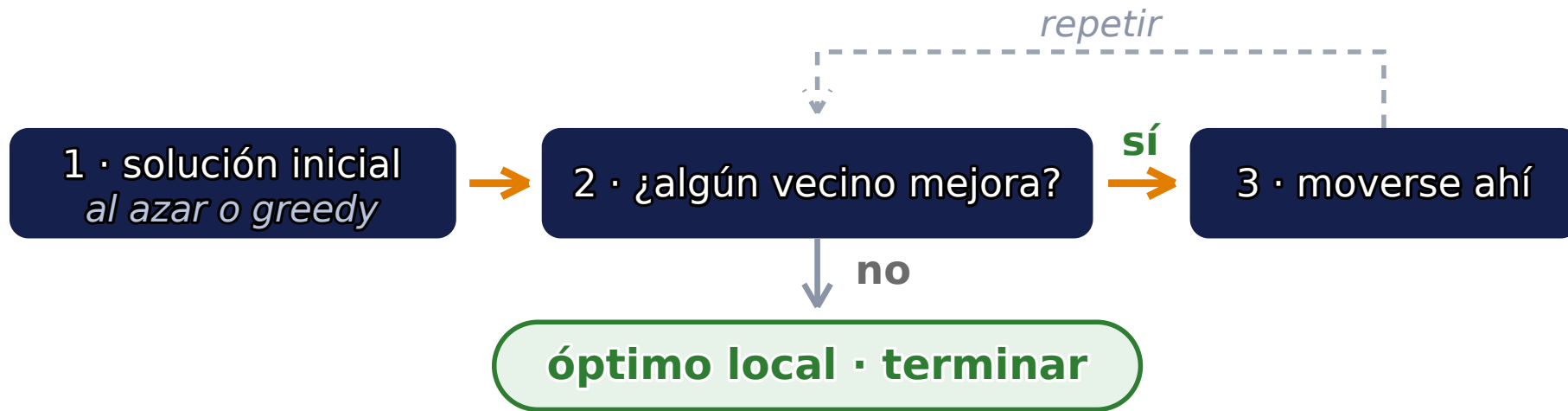
Hill Climbing: subir en la niebla



Subir siempre termina en la **primera cima**: el óptimo local, no el global.



Hill Climbing: el algoritmo



ejemplo · la permutación de costo mínimo

2 4 1 3 5



solución = un orden

vecino = intercambiar dos posiciones

altura = $-\text{costo}$

El bucle solo **sube**: se detiene en el primer punto sin vecino mejor.

Hill Climbing: dónde se atasca

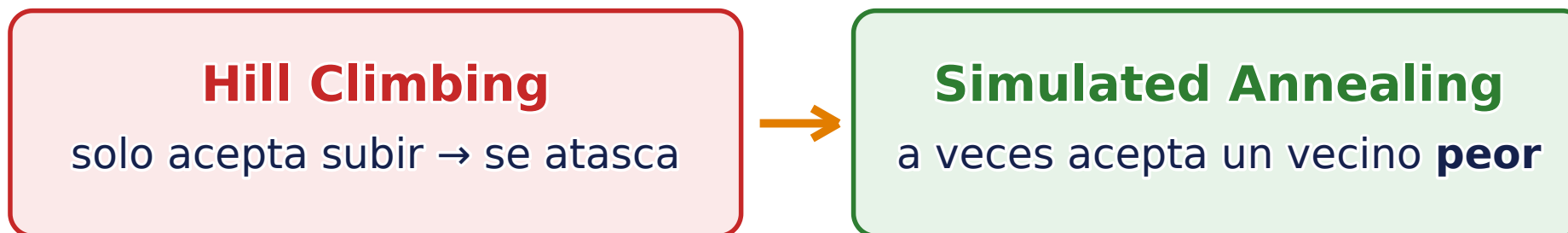


Solo subimos: la búsqueda muere en la **primera cima**, quizá lejos del óptimo global.

¿Cómo le permitimos **bajar** a veces, para escapar de la loma?



De Hill Climbing a Simulated Annealing



¿Cuánto peor? Lo controla la **temperatura T**

T alta · explora

acepta casi cualquier paso tiempo →

T baja · explota

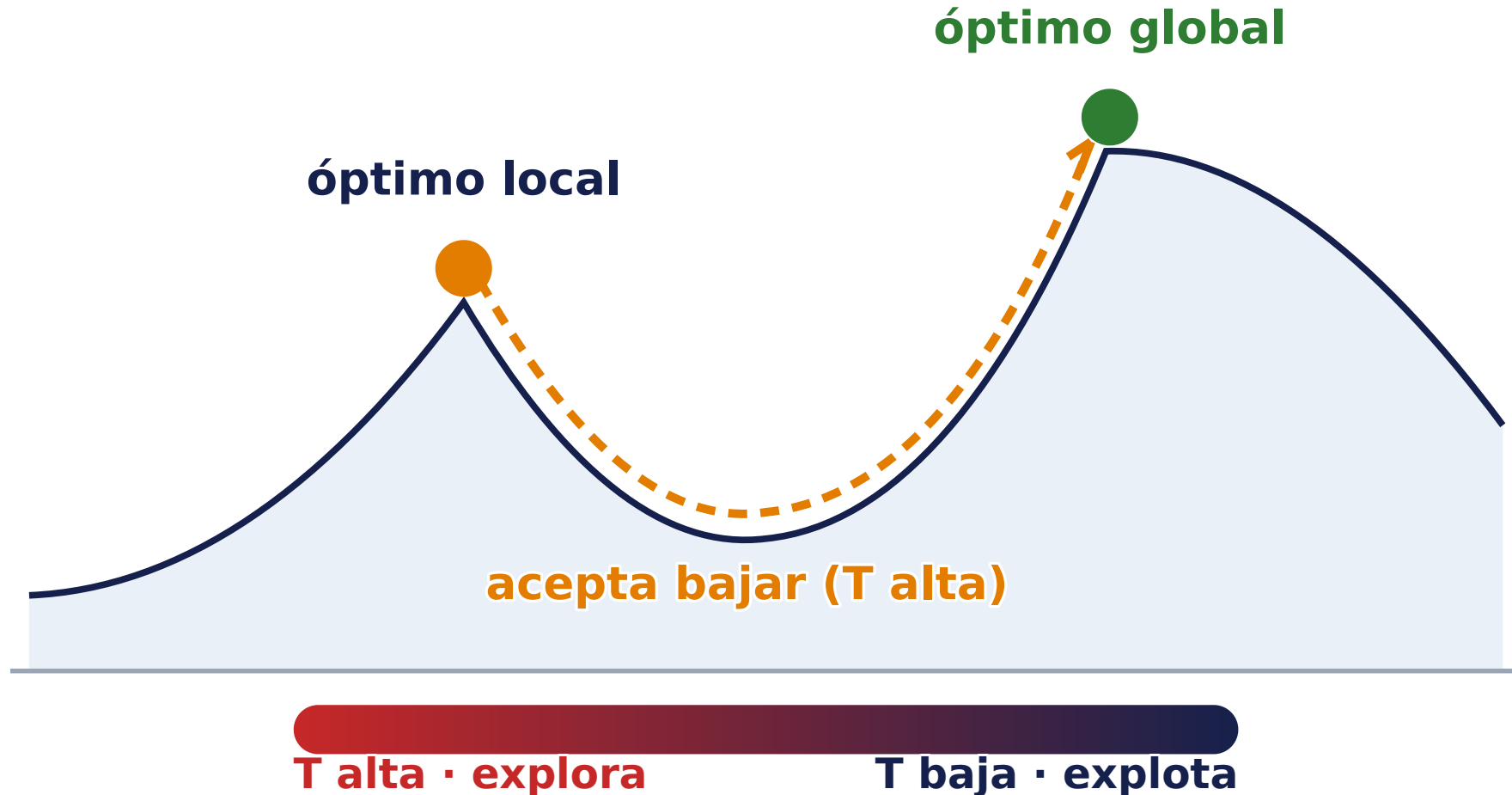
casi solo mejoras

«recocido» físico: enfriar un metal despacio → sus átomos se asientan en mínima energía

Hill climbing es el caso $T = 0$: nunca acepta empeorar.



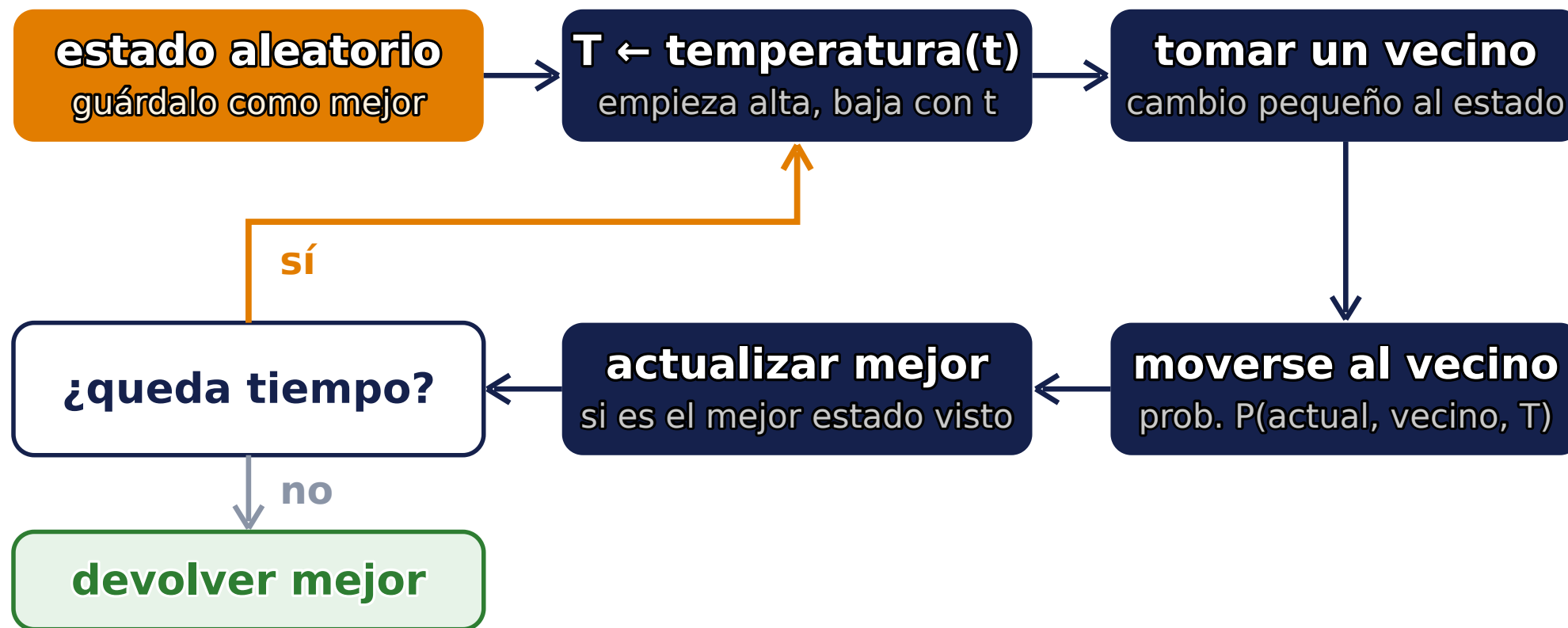
Simulated Annealing: bajar para cruzar



Aceptar a veces un vecino **peor** ($P = e^{-\Delta/T}$) permite cruzar el valle.



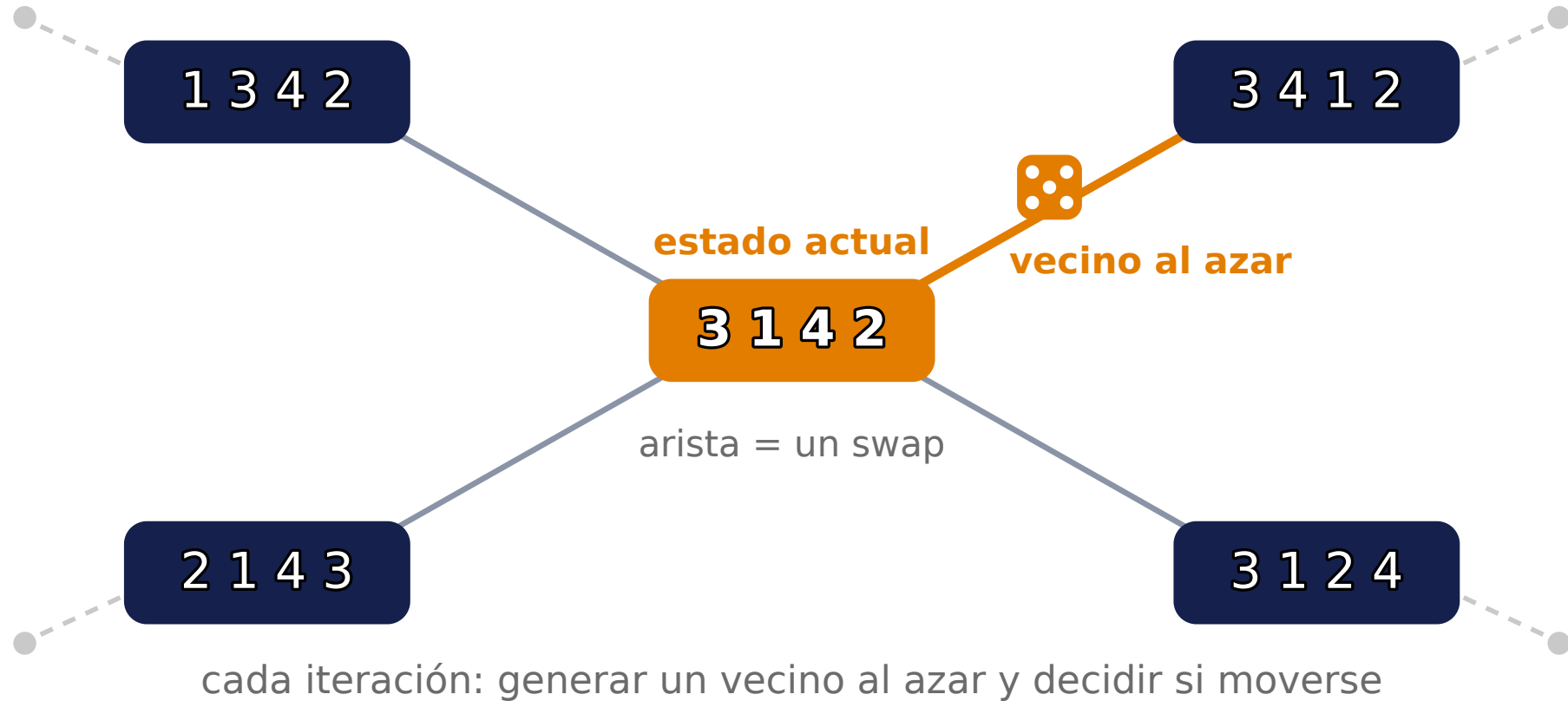
Simulated Annealing: el bucle



Quedan **tres piezas** por definir: el **vecino**, la **temperatura** $T(t)$ y la **aceptación** P .



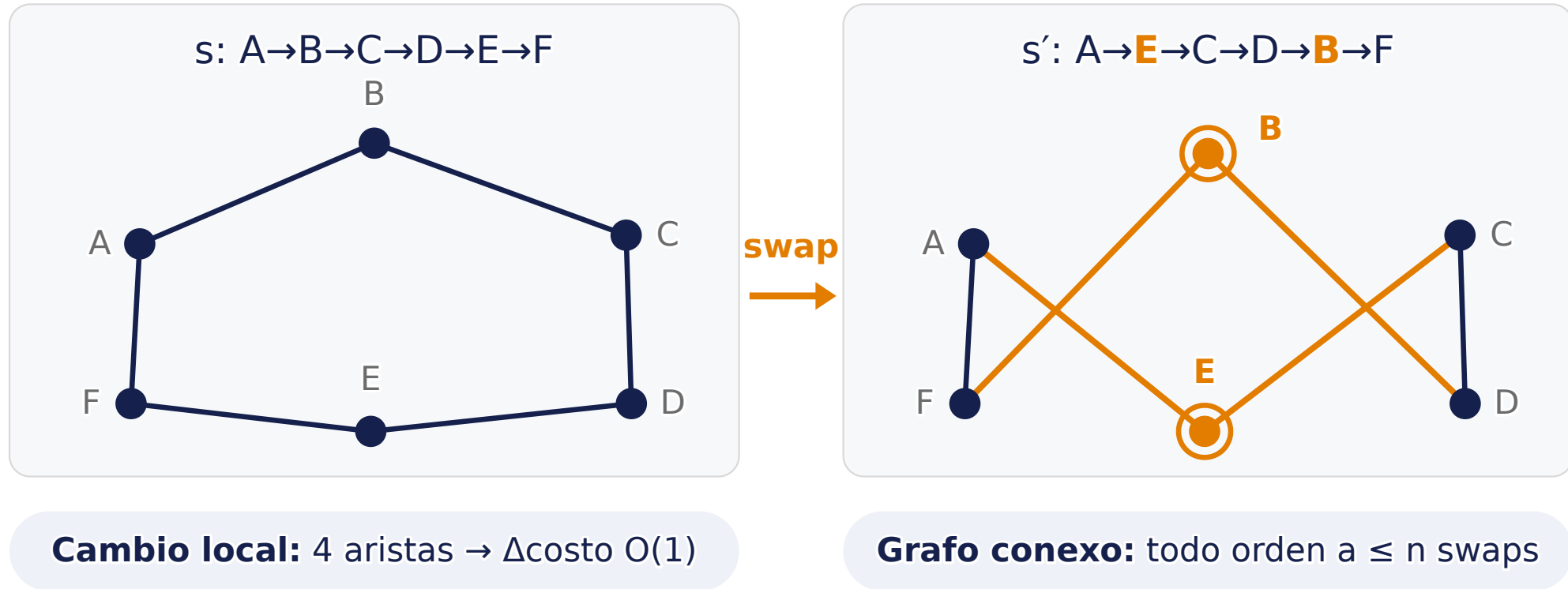
SA: la función vecino



Nodo = solución, arista = un cambio pequeño: **búsqueda local = caminar por este grafo**, sin construirlo jamás.



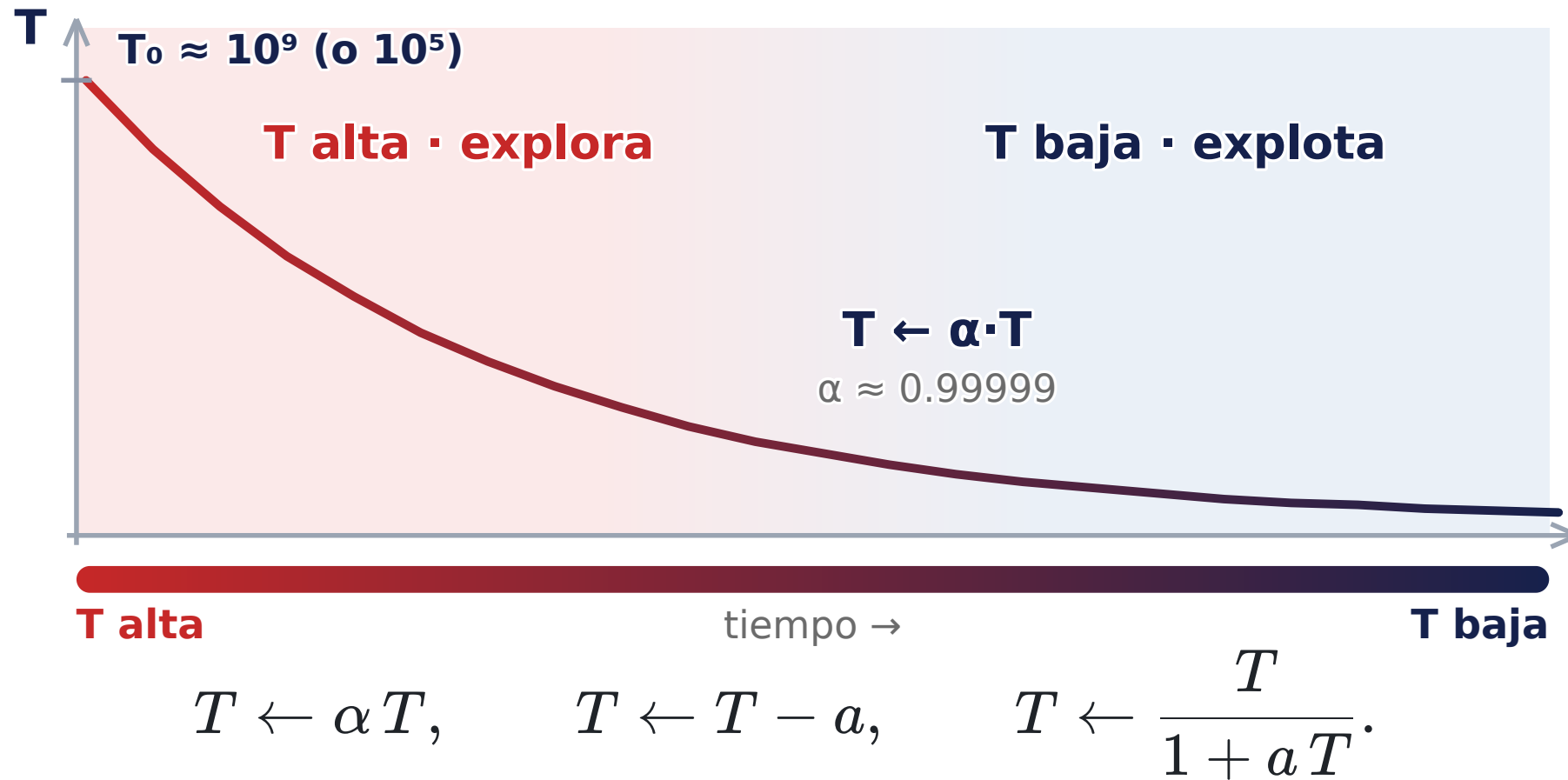
SA: vecino en el TSP



Un **vecino** sale de un cambio pequeño: en TSP, intercambiar dos ciudades del orden.



SA: la temperatura

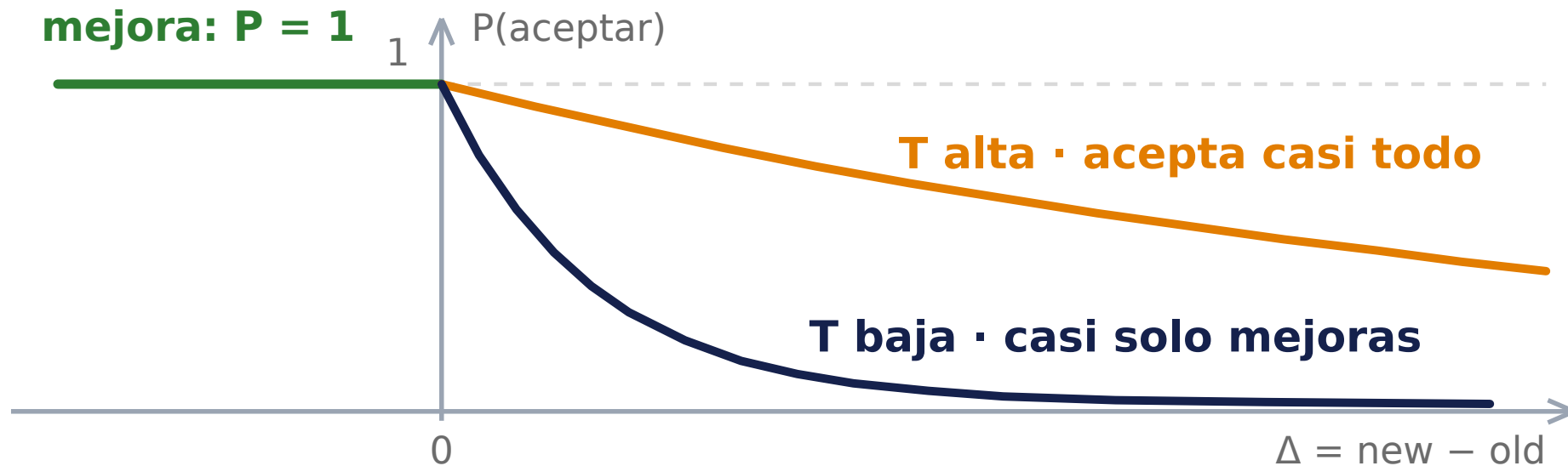


Enfriamientos comunes; en la práctica, casi siempre la **geométrica**.



SA: probabilidad de aceptación

$$P(\text{old}, \text{new}, T) = \begin{cases} 1 & \text{si } \text{new} < \text{old}, \\ e^{-\Delta/T}, & \Delta = \text{new} - \text{old} \quad \text{si } \text{new} \geq \text{old}. \end{cases}$$

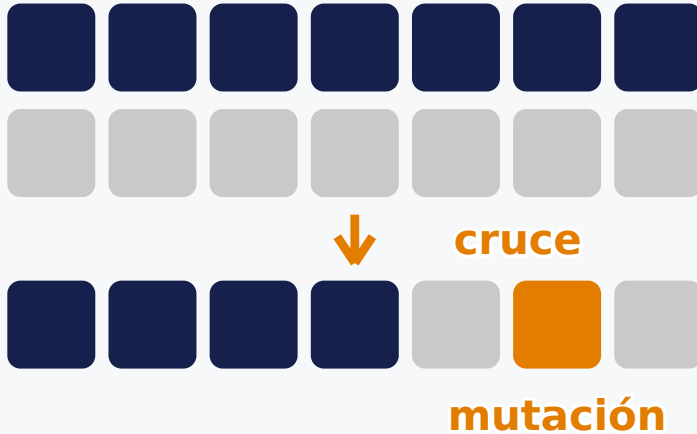


Criterio de **Metropolis**. **Heurística**: no garantiza el óptimo ni verifica al final.



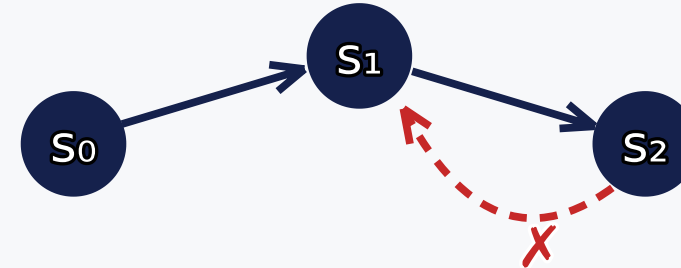
Otras metaheurísticas

Algoritmos genéticos



población: combinar y perturbar soluciones

Búsqueda tabú



lista tabú: movimientos recientes

prohibir volver → salir del óptimo local

Misma idea, otras formas de explorar el espacio.

Cuál conviene **depende del tipo de contest** (rondas heurísticas largas); en lo personal, no las he usado en competencia.



Plantilla de RNG en C++

El único código de la clase. La semilla va del reloj: cada ejecución usa tiradas distintas:

```
#include <random>
#include <chrono>
using namespace std;

mt19937_64 rng(
    chrono::steady_clock::now().time_since_epoch().count());

long long rnd(long long l, long long r) {
    return uniform_int_distribution<long long>(l, r)(rng);
}
```

Evitar `rand()` (rango corto, baja calidad) y las semillas constantes. Para barajar:
`shuffle(v.begin(), v.end(), rng)`.



Señales para usar el azar

Hay muchas soluciones válidas

una fracción p del espacio sirve

muestrear y verificar · $E = 1/p$

mayoritario · Farming Mars · CSES 3422

Verificar es más barato

que recomputar la respuesta completa

comparar huellas aleatorias

Freivalds · hashing XOR · Power of Three

Espacio gigante, sin estructura

la respuesta está cerca de una random

hill climbing · Simulated Annealing

rondas heurísticas (AHC)

Tu peor caso es un input fijo

una elección fija siempre pierde con él

pivote y semilla al azar

quicksort · plantilla RNG

Son **señales**, no un árbol de decisión: en la práctica se mezclan, y lo que queda es **probar**.

¡Fin!